

Lab 3: Component Modelling & Architectural Pattern Selection

Problem Statement #17 — Vaccination Cohort & Dose Scheduling System

Name: Zayaan Ahmed

SRN: PES1UG24AM457

Deliverable 1: Six-Step Architecture Analysis & UML Component Diagram

Step 1: Scenario Review

The system supports citizens booking vaccination slots and vaccination officers administering doses and managing certificates. The architecture must preserve booking consistency, enforce dose rules, and support secure, responsive certificate verification.

Key Functional Requirements

- Citizen Registrant can book a slot for a selected center, dose number, date and time window.
- The system checks real-time slot availability before confirming a booking.
- Dose-interval eligibility is verified before later-dose booking is allowed.
- Vaccination Officers can record administered doses and generate digitally signed QR certificates.
- Vaccination certificates can be verified by QR scan and digital-signature validation.

Key Non-Functional Requirements and Challenges

- Performance: certificate verification should complete in under 150 ms under simulated peak load.
- Reliability and consistency: concurrent booking requests must not create double-bookings; target uptime is 99.5% during active drive hours.
- Security: protect citizen health data, restrict officer-only actions, and reject forged or altered certificates.
- Usability: booking and officer workflows should remain simple during high-demand vaccination drives.

Step 2: Architectural Style Analysis

Architecture	Structure	Advantages for this system	Limitations for this system	Decision
Layered	Presentation → Business → Data	Clear separation of concerns; easy testing and maintenance; transaction logic can remain centralized.	Individual modules are less independently scalable than microservices.	Selected
Microservices	Small independently deployable services	Independent scaling and fault isolation; useful for very large national-scale deployments.	More operational complexity, network latency, and distributed consistency problems for atomic booking.	Not selected
Client-Server	Central server serving multiple clients	Simple deployment and centralized data consistency.	Server can become a bottleneck/single point of failure and gives less internal modular structure.	Not selected

Step 3: Architecture Selection & Component Identification

Selected architecture: Layered Architecture. It directly separates the user interfaces, vaccination business rules, and persistent data while keeping the booking transaction within a single application boundary.

Layer	Component	Responsibility
Presentation	Citizen Portal	Searches slots, submits bookings, views confirmations/certificates.
Presentation	Vaccination Officer Portal	Records administered doses and performs certificate operations.
Business Logic	Booking & Scheduling	Checks/rechecks capacity, reserves slots atomically, generates booking references.
Business Logic	Eligibility & Cohort Rules	Validates minimum dose interval and cohort eligibility.
Business Logic	Vaccination Record & Certificate	Stores dose administration, creates signed QR certificates, verifies signatures.
Data Access / Persistence	Vaccination Data Repository	Stores citizen, center, slot, dose and certificate data.

Step 4: Component Diagram Creation

The diagram below uses component stereotypes, logical layer grouping, and labelled service/data interfaces. Required interfaces are represented at the consuming side and provided interfaces at the component offering the service.

Consumer	Provider	Interface / Data Flow
Citizen Portal	Booking & Scheduling	Booking API — slot search, booking request and confirmation
Vaccination Officer Portal	Vaccination Record & Certificate	Dose & Certificate API
Booking & Scheduling	Eligibility & Cohort Rules	Eligibility API — dose interval/cohort validation
Booking & Scheduling	Vaccination Data Repository	Slot queries and atomic reservation transaction
Eligibility & Cohort Rules	Vaccination Data Repository	Citizen demographic and prior-dose query
Vaccination Record & Certificate	Vaccination Data Repository	Dose record and certificate data access

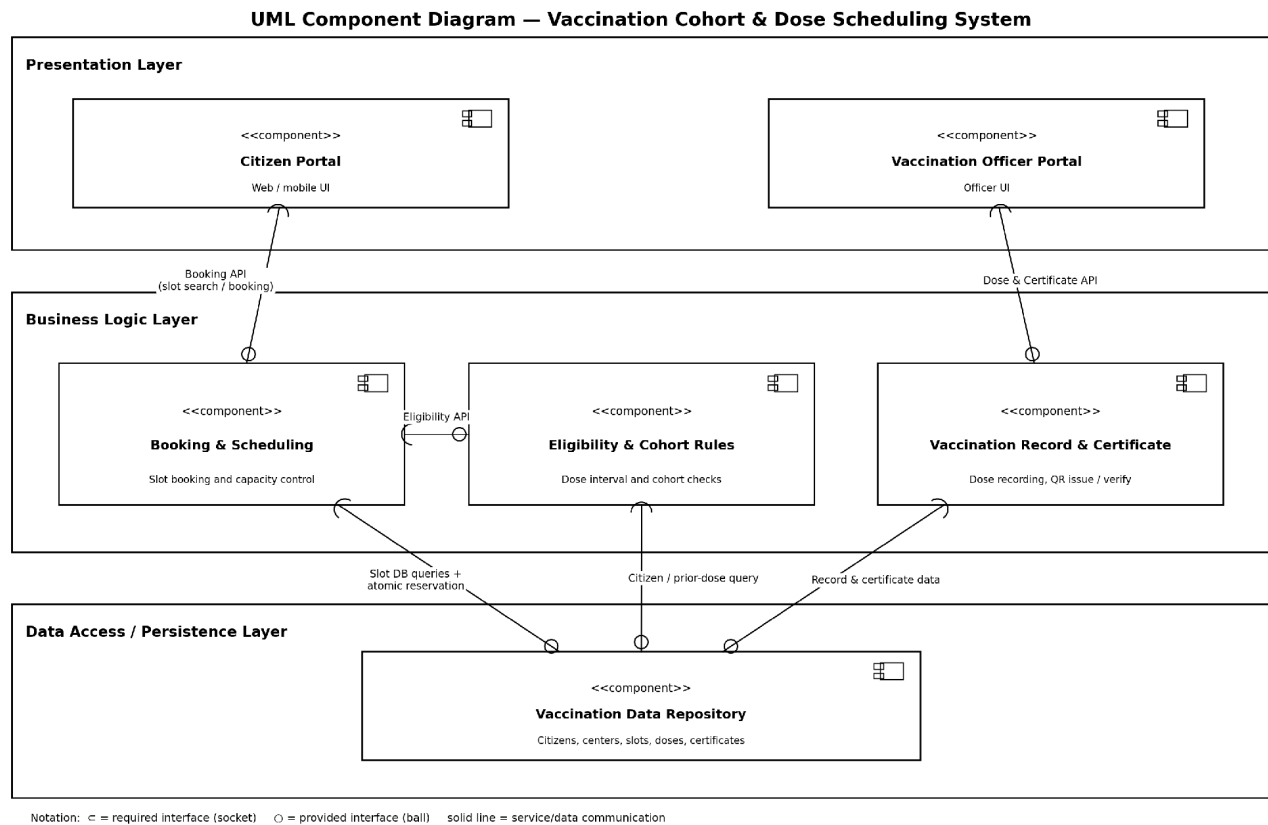


Figure 1. UML Component Diagram for the Vaccination Cohort & Dose Scheduling System

Step 5: Written Justification Preparation

A separate one-page architectural justification has been prepared as Deliverable 2 below. It states the selected architecture, two scenario-specific reasons, one security advantage, and one performance benefit, as required by the lab handout.

Step 6: Diagram Finalization & Export

- All six identified components are present and grouped into Presentation, Business Logic, and Data Access/Persistence layers.
- Six labelled interfaces are shown, exceeding the minimum requirement of four.
- Component stereotypes and provided/required interface notation are included.
- Interface directions and dependencies were reviewed against the booking, eligibility, recording and certificate workflows; the final diagram is embedded in this document for submission.

Deliverable 2: Written Architecture Justification

Architecture Selection

We chose Layered Architecture for the Vaccination Cohort & Dose Scheduling System. The system is organized into a Presentation Layer for Citizen Registrant and Vaccination Officer interfaces, a Business Logic Layer for booking, slot-availability checks, dose-interval validation, vaccination records and certificate processing, and a Data Access Layer for citizen, center, slot and vaccination data.

Reason 1 — Clear Separation of Responsibilities

The system contains distinct user-interface, rule-processing and data-management responsibilities. Keeping these concerns in separate layers makes the design easier to understand, test and maintain. For example, a change in dose-interval eligibility rules can be made in the Business Logic Layer without changing the citizen interface or the database implementation.

Reason 2 — Strong Transaction and Data Consistency

Slot booking is concurrency-sensitive: availability must be rechecked when the citizen submits the request, and the slot reservation and capacity decrement must occur atomically so that two users cannot obtain the same final slot. A layered design keeps this transaction logic within one application boundary, making consistency easier to enforce than a distributed microservices design that would require coordination across multiple services.

Security Advantage

The layered design prevents clients from directly accessing the database. Authentication, role-based authorization and validation can be enforced in the Business Logic Layer before data operations are performed. Vaccination Officers can be restricted to privileged functions such as recording administered doses and issuing certificates, while digital-signature validation can be centralized for QR-code certificate verification.

Performance Benefit

For the current system scope, layered architecture avoids the extra network calls and serialization overhead that would come from splitting booking, eligibility, availability and certificate verification into separate microservices. Local service calls and a controlled data-access layer reduce latency and simplify optimization of frequently used operations, supporting fast slot checks and near-instant certificate verification.

Why Microservices Were Not Selected

Microservices provide independent scaling and fault isolation, but they also introduce deployment complexity, network latency and distributed data-consistency problems. These costs are not justified for the present scope, where the main functions are closely related and share transaction-sensitive vaccination data. If the platform later grows to national scale, selected high-load modules such as certificate verification or notifications could be separated into services.